© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

Duración: 2h15m **Nombre:**

1. (1.0 ptos) ¿La siguiente afirmación es cierta o falsa? Demuéstralo.

$$2\,(lg\sqrt{n^3}) + \sqrt{n} \in \theta(\sqrt[3]{n})$$

2. (1.5 ptos)

```
func f(a: vector[1..n][1..n] de Natural) dev a: vector[1..n][1..n]
para i ← 1 hasta n-1
   j ← i+1
   aux ← a[i][j]
   mientras a[i][j] ≤ a[i][i] ∧ j ≤ n
      aux ← aux + a[i][j]
      j ← j+1
   fmientras
   a[i][j] ← aux
fpara
```

a) Para n = 3, escribe 2 matrices distintas que correspondan a la instancia mejor.

b) Calcula el coste exacto en pasos de los casos mejor y peor.

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

c) Coste asintótico de la función.

3. (1.0 ptos) Completa la siguiente función recursiva. Ten en cuenta que es posible llamarla con un vector vacío (N = 0).

```
func f(v: Vector[1..N] de Real; i: Natural; x: Real) dev res: Bool

si Cond
    res ← E1
sino
    res ← f(v, E2, x)
    res ← E3

{Post: res = ∀α: i ≤ α ≤ N: v[α] = x}
```

Cond:
E1:
E2:
E3:

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

4. (1.25 ptos) Calcula el **coste exacto** en pasos y el **coste asintótico** de la siguiente función recursiva. (Resolver en última página)

```
func f(n: N) dev s: N
si n < 1
      s ← 0
sino
      s ← 0
      para i ← 1 hasta 2
            s ← s + f(n-1)
            para i ← 1 hasta n
                  s ← s * i
            fpara
      fpara
fsi
```

5. (1.0 ptos) Escribe la especificación de las siguientes funciones, incluyendo precondición, postcondición y la definición de **operaciones** auxiliares (con sus ecuaciones) en caso de ser necesarias.

a) Una función que devuelva la **posición** del máximo de un vector.

b) Una función que devuelva el elemento que está en la **base** de una pila.

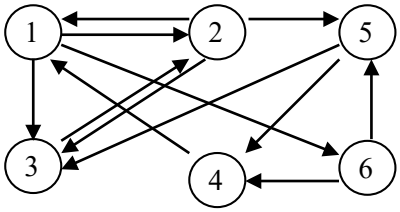
© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

6. (1.0 pto) Implementa el método ***altura*** de la clase ABB, que devuelve la altura del árbol. Se pueden implementar también métodos auxiliares si se considera necesario. La definición de la clase ABB es la siguiente:

```
class ABB
{
public:
    typedef int TipoClave;
    typedef float TipoDato;
    ...
    unsigned altura() const;
private:
    class NodoBin
    {
    public:
        TipoClave clave;
        TipoDato  dato;
        NodoBin * izq;
        NodoBin * der;
            NodoBin(): izq(NULL), der(NULL) {};
            NodoBin(TipoClave c, TipoDato d)
            : clave(c), dato(d), izq(NULL), der(NULL) {};
            ~NodoBin();
    };
    typedef NodoBin * PtrNodoBin;
    PtrNodoBin miRaiz;
};
```

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

7. (1.0 ptos) Dado el siguiente grafo.



a) Representa el grafo mediante una **lista de adyacencia**.

b) Realiza una **búsqueda en profundidad** del grafo empezando en el nodo 1. Dibuja el árbol que representa el recorrido e indica el orden en que se visitarían los nodos.

Árbol:

Nodos:

8. (1.0 ptos)

a) Dado el siguiente resultado del algoritmo de **Prim**, escríbelo en forma de grafo.

	1	2	3	4	5	6	7
vecino:	1	3	1	3	1	2	6

b) Dado el siguiente resultado del algoritmo de **Dijkstra** donde el nodo inicial era el 1, escribe los caminos más cortos desde el nodo 1 a todos los demás.

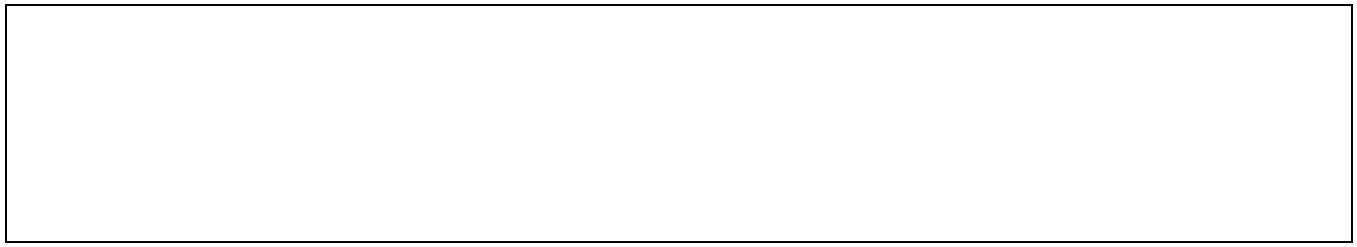
	1	2	3	4	5	6	7
vecino:	1	3	1	3	1	2	5

	1	2	3	4	5	6	7
pvecino:	0	4	2	3	3	5	5

De 1 a 2:
De 1 a 3:
De 1 a 4:
De 1 a 5:
De 1 a 6:
De 1 a 7:

9. (1.25 ptos) Escribe un algoritmo de backtracking que permita colorear un grafo con el mínimo de colores posibles. Los colores se representan con los valores del 1 al n, 0 representa que no hay color. Recuerda que al colorear un grafo, los nodos vecinos han de tener colores distintos. Si tu algoritmo utiliza alguna función auxiliar, impleméntala también.

```
ALGORITMO Colorea(i:Nat; max color: Nat)
DATOS:      arco: vector[1..n][1..n] de Bool;  // Matriz de adyacencia
RES/GLOBAL:      colorM: vector[1..n] de Nat  // Vector resultado
                 max_colorM: Nat;     // N° colores resultado
GLOBAL:     color: vector[1..n] de Nat  // Solución parcial
AUX:        c: Conjunto de Nat    // Colores de los vecinos
INICIO: max_colorM ← Inf   // primera llamada Colorea(0,0)
METODO:
Si i=n entonces
```



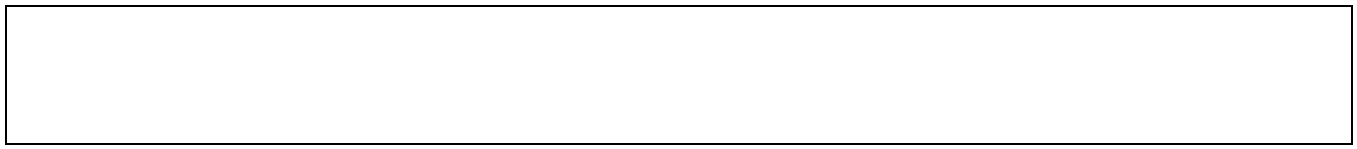
```
sino
  c ← ∅     // Creamos el conjunto de colores de los vecinos
  para j ← 1 hasta n
    si
      c ← c ∪ {color[j]}
    fsi
  fpara
  para j ←
```



```
    si
```



```
      Colorea(
```



```
  fpara
fSi
```